

Relatório de Desenvolvimento — FitLife

Do laboratório inicial ao CRUD responsivo final

Projeto FitLife • versão final • 13/09/2026

1. Visão geral

O FitLife foi desenvolvido como um sistema de gerenciamento de alunos de academia. O escopo evoluiu de um cadastro simples para uma aplicação CRUD com validações, persistência local, busca, edição, exclusão, interface planejada no Figma, responsividade e versionamento com Git/GitHub.

Status final: FitLife v1 concluído e repositório sincronizado com a branch main.

2. Requisitos implementados

- RF01 — cadastrar nome do aluno.
- RF02 — cadastrar idade.
- RF03 — selecionar plano.
- RF04 — selecionar status.
- Cadastrar, listar, editar e excluir alunos (CRUD).
- Buscar aluno pelo nome, inclusive por trecho e sem diferenciar maiúsculas/minúsculas.
- Mostrar novamente todos os alunos após uma busca.
- Persistir os dados com localStorage.
- Interface desktop e mobile responsiva.

3. Regras e validações

- Nome obrigatório e validado para aceitar letras/espços.
- Idade obrigatória, inteira, mínima de 16 e máxima de 70.
- Plano obrigatório.
- Status obrigatório.
- Após cadastro/edição bem-sucedidos, o formulário é limpo.
- Idades 16/17 foram tratadas no contexto do laboratório com necessidade de autorização; abaixo de 16, cadastro recusado.

4. CRUD e persistência

Os alunos são mantidos em um array carregado do localStorage. Objetos de aluno reúnem nome, idade, plano e status. O índice de edição é controlado separadamente para diferenciar criação e atualização.

- Create: adiciona um novo objeto ao array.
- Read: renderiza a lista de alunos.
- Update: carrega o aluno no formulário e substitui o item correto ao salvar.

- Delete: remove o aluno pelo índice e persiste a alteração.
- Persistência: JSON.stringify ao salvar e JSON.parse ao carregar.
- Teste com F5 confirmou permanência dos dados.

5. Busca e refatoração final

A busca começou com um HTML próprio e visual diferente da listagem principal. Na revisão final, a renderização foi centralizada em renderizarAlunos(lista), permitindo que a listagem completa e os resultados filtrados usem os mesmos cards.

- buscarAluno() ficou responsável por filtrar; renderizarAlunos() ficou responsável por apresentar.
- O HTML duplicado da busca foi removido.
- Foi usado alunos.indexOf(aluno) para recuperar o índice original do array e preservar Editar/Excluir após uma busca filtrada.
- Foram testadas busca completa, busca parcial e ações de edição/exclusão no resultado filtrado.

6. UI/UX e conversão Figma → HTML/CSS

A interface foi desenhada antes da implementação visual. O desktop usa formulário e listagem lado a lado; o mobile empilha os setores e transforma a tabela em cards.

- Paleta principal: azul #2563EB, superfícies claras e textos em tons slate.
- Inputs/selects com 48px de altura, borda #CBD5E1 e raio 8px.
- Cards brancos com borda #E2E8F0 e raio 12px.
- Cadastrar e Buscar como ações primárias; Mostrar Todos como secundária.
- Editar neutro; Excluir com semântica vermelha.
- Badges Ativo (verde) e Inativo (laranja).

7. Responsividade

A tabela desktop não foi simplesmente comprimida. No mobile, cada registro se torna um card: nome e status no topo, idade/plano em uma linha resumida e botões dividindo a largura.

- Media query principal: até 768px.
- Cabeçalho da tabela escondido no mobile.
- aluno-info aparece somente no mobile; idade/plano separados ficam no desktop.
- display: contents foi usado no desktop para preservar a ordem visual das colunas; display: flex no mobile cria o topo nome/status.
- Testes visuais em 375px e 320px passaram sem overflow horizontal.
- Nome longo quebrou corretamente sem deslocar o badge.
- Header mobile recebeu padding lateral coerente com os cards.

8. Bugs, diagnósticos e aprendizados

Problema	Causa/diagnóstico	Solução/aprendizado
Alert ao atualizar página	Evento/fluxo disparava indevidamente.	Vincular validação/cadastro ao clique correto.
Option padrão dos selects	Valor padrão não representava ausência de escolha.	Usar placeholder com <code>value=""</code> .
Alerts duplicados	Fluxo de validação continuava após erro.	Organizar condições e retornos.
Aluno duplicado ao editar	<code>push()</code> continuava sendo executado no fluxo de atualização.	Separar criação de atualização pelo <code>indiceEditando</code> .
Listagem duplicada	HTML era anexado manualmente e também por <code>mostrarAlunos()</code> .	Centralizar a renderização.
<code>buscarAluno</code> não definida	Função ficou aninhada por fechamento de chave incorreto.	Revisar escopo e chaves.
Hover parecia não funcionar	Arquivos não mostraram conflito; após atualização da página passou a funcionar.	Validar navegador/DevTools antes de alterar CSS no chute.
Badge ocupava largura excessiva	Largura da coluna estava no próprio elemento visual.	Separar container da coluna e <code>span</code> do badge.
Mobile: status abaixo do nome	Conflito entre <code>display: contents</code> e <code>display: flex</code> / ordem da cascata.	Reorganizar regras padrão e media query; validar cascata.
Desktop alterado pelo wrapper mobile	Nome/status deixaram de ser filhos visuais diretos do flex principal.	Usar <code>display: contents</code> no desktop e <code>flex</code> no mobile.
Busca com visual antigo	<code>buscarAluno()</code> tinha HTML duplicado.	Criar <code>renderizarAlunos(lista)</code> e reutilizar.

9. Uso de IA/Codex no desenvolvimento

O Codex foi usado como apoio pontual, não como substituto do processo de aprendizagem. Em um caso, analisou o hover sem alterar arquivos; em outro, corrigiu o conflito responsivo depois que o problema já havia sido isolado e o objetivo visual estava definido.

- Uso como revisor/diagnosticador de código.
- Pedidos delimitados: preservar CRUD, fazer a menor alteração possível e explicar a causa.
- Validação manual no navegador depois da alteração.
- Aprendizado: IA é mais útil quando recebe contexto, objetivo e limites claros.

10. Git e histórico da reta final

Commit	Descrição
47eee08	feat: estiliza botao principal do formulario
8fc41be	feat: estiliza acoes de busca do formulario
94e62bf	feat: organiza estrutura da listagem de alunos
a0a32ec	feat: estiliza acoes da listagem de alunos
2cf1e9c	feat: inicia logica de status dos alunos
debc16c	feat: adiciona badges de status na listagem
4724104	feat: inicia responsividade da listagem de alunos
3e3dcea	feat: finaliza responsividade dos cards de alunos
aa722f9	refactor: reutiliza renderizacao na busca de alunos
2cb3c87	chore: remove logs de depuracao

11. Testes finais executados

- Cadastro de novo aluno e exibição imediata.
- Edição de dados e troca de status sem duplicação.
- Busca por nome e por trecho.
- Editar/Excluir a partir de resultado filtrado com índice original correto.
- Mostrar Todos restaura a lista completa e limpa a busca.

- Exclusão remove o registro correto.
- F5 preserva dados via localStorage.
- Desktop revisado após alterações mobile.
- Mobile validado em 375px e 320px.
- console.log de depuração removidos antes do encerramento.

12. Encerramento

A versão final encerra a evolução do laboratório com as funcionalidades planejadas operacionais e uma interface coerente entre desktop e mobile. O projeto também consolidou um processo de trabalho: implementar em etapas pequenas, testar, investigar erros com DevTools, registrar aprendizados, versionar checkpoints e refatorar quando a duplicação começou a prejudicar a manutenção.

FitLife v1 — concluído com sucesso.